

Санкт-Петербургский политехнический университет Петра
Великого

Кафедра : Высшей школе прикладной математики и
вычислительной физики

ОПИСАНИЕ ФОРМАТА ЗАКОДИРОВАННОГО ФАЙЛА

АЛГОРИТМ ХАФФМАНА

Техническая документация

Выполнила:
студентка группы
5030102/40002
Михайленко В.Д.

Санкт-Петербург
2026

Аннотация

Настоящий документ содержит описание формата файла, получаемого в результате сжатия данных статическим алгоритмом Хаффмана.

Ключевые слова: Хаффман, сжатие данных, кодирование, декодирование, словарь, битовый поток, префиксный код.

Основные характеристики формата:

- Самодостаточность – словарь встроен в файл;
- Минимальный объём служебных данных – $4 + N$ байт (N – количество уникальных символов);
- Поддержка любых байтовых значений (0-255);
- Восстановление данных без потерь.

Содержание

Аннотация	1
1 Общая структура файла	3
2 Заголовок файла	3
3 Словарь	3
3.1 Структура записи	3
3.2 Упаковка кода в байты	3
3.3 Чтение словаря	4
4 Длина закодированных данных	4
5 Закодированные данные	4
6 Примеры	4
6.1 Пример 1: 10 одинаковых символов	4
6.2 Пример 2: 3 разных символа	5
6.3 Пример 3: Бинарный файл Main.class	5
7 Алгоритм декодирования	6
7.1 Основной алгоритм	6
7.2 Функция побитового чтения	6
8 Характеристики формата	7
8.1 Преимущества	7
8.2 Недостатки	7
8.3 Область оптимального применения	7
9 Заключение	7

1 Общая структура файла

Файл состоит из четырёх основных частей:

Размер	Тип	Описание
4 байта	int	Количество уникальных символов (N)
Переменный	словарь	N записей
4 байта	int	Количество битов в закодированных данных
Переменный	данные	Закодированная последовательность

Таблица 1: Общая структура сжатого файла

2 Заголовок файла

Первые 4 байта содержат количество уникальных символов (N) в формате big-endian.

```
1 int dictSize = dis.readInt();  
2
```

Листинг 1: Чтение заголовка

Примеры:

- 0x00000001 – 1 символ
- 0x00000003 – 3 символа

3 Словарь

Словарь состоит из N записей.

3.1 Структура записи

Размер	Тип	Описание
1 байт	byte	Исходный символ (0-255)
1 байт	byte	Длина кода в битах (1-255)
Переменный	биты	Код Хаффмана (MSB first)

Таблица 2: Структура записи словаря

3.2 Упаковка кода в байты

Код хранится в формате MSB (первый бит – самый левый).

- Код "101" (3 бита) -> байт: 10100000 = 0xA0
- Код "1100" (4 бита) -> байт: 11000000 = 0xC0

3.3 Чтение словаря

```
1 Map<String, Byte> reverseCodes = new HashMap<>();
2 for (int i = 0; i < dictSize; i++) {
3     byte symbol = dis.readByte();
4     int len = dis.readByte();
5     String code = readBits(dis, len);
6     reverseCodes.put(code, symbol);
7 }
8
```

Листинг 2: Чтение словаря

4 Длина закодированных данных

Следующие 4 байта указывают точное количество битов в закодированных данных (не байтов).

```
1 int bitLength = dis.readInt();
2
```

Листинг 3: Чтение длины данных

Примеры:

- 0x0000000A – 10 битов
- 0x0000001E – 30 битов

5 Закодированные данные

Последняя часть файла содержит закодированные данные – последовательность кодов Хаффмана, упакованных в байты.

```
1 String encodedData = readBits(dis, bitLength);
2
```

Листинг 4: Чтение закодированных данных

6 Примеры

6.1 Пример 1: 10 одинаковых символов

Исходный файл: "1111111111" (10 байт)

- Символ '1' (0x31), частота 10, код "0"

Структура сжатого файла:

Заголовок: 0x00000001
Словарь: 0x31 0x01 0x00
Длина данных: 0x0000000A (10 бит)
Данные: 0x00 0x00

Итоговый размер: $4 + 3 + 4 + 2 = 13$ байт

6.2 Пример 2: 3 разных символа

Исходный файл: "11111111112222233333" (20 байт)

Частоты символов:

- '1': 10 раз
- '2': 5 раз
- '3': 5 раз

Коды Хаффмана:

- '1' $\rightarrow$ "0" (1 бит)
- '2' $\rightarrow$ "10" (2 бита)
- '3' $\rightarrow$ "11" (2 бита)

Структура сжатого файла:

Заголовок: 0x00000003
Словарь: 0x31 0x01 0x00
0x32 0x02 0x80
0x33 0x02 0xC0
Длина данных: 0x0000001E (30 бит)
Данные: 0x00 0xAA 0xAF 0xF0

Итоговый размер: $4 + 9 + 4 + 4 = 21$ байт

6.3 Пример 3: Бинарный файл Main.class

- Исходный размер: 6659 байт
- Сжатый размер: 6042 байта
- Коэффициент сжатия: 1.10
- Сжатие: 9.27%

7 Алгоритм декодирования

7.1 Основной алгоритм

```
1 public void decompress(InputStream in, OutputStream out) {
2     int dictSize = readInt(in);
3
4     Map<String, Byte> map = new HashMap<>();
5     for (int i = 0; i < dictSize; i++) {
6         byte symbol = readByte(in);
7         int len = readByte(in);
8         String code = readBits(in, len);
9         map.put(code, symbol);
10    }
11
12    int bitLen = readInt(in);
13    String data = readBits(in, bitLen);
14
15    StringBuilder cur = new StringBuilder();
16    for (char bit : data.toCharArray()) {
17        cur.append(bit);
18        if (map.containsKey(cur.toString())) {
19            out.write(map.get(cur.toString()));
20            cur = new StringBuilder();
21        }
22    }
23 }
24
```

Листинг 5: Алгоритм декодирования

7.2 Функция побитового чтения

```
1 private String readBits(InputStream in, int count) {
2     int bytes = (count + 7) / 8;
3     byte[] buf = new byte[bytes];
4     in.read(buf);
5
6     StringBuilder bits = new StringBuilder();
7     for (int i = 0; i < count; i++) {
8         int byteIdx = i / 8;
9         int bitIdx = 7 - (i % 8);
10        int bit = (buf[byteIdx] >> bitIdx) & 1;
11        bits.append(bit);
12    }
13    return bits.toString();
14 }
```

8 Характеристики формата

8.1 Преимущества

- Самодостаточность – словарь встроен в файл;
- Минимальный объём служебных данных – $4 + N$ байт;
- Поддержка любых байтов (0-255);
- Сжатие без потерь.

8.2 Недостатки

- Неэффективно для файлов размером менее 20 байт;
- Требуется побитовый ввод/вывод.

8.3 Область оптимального применения

- Файлы размером более 1 КБ;
- Данные с неравномерным распределением символов;
- Текстовые файлы, исходный код, XML/JSON.

9 Заключение

Данный формат реализует статическое кодирование Хаффмана:

1. Дерево Хаффмана строится один раз для всего файла;
2. Словарь встраивается в начало файла;
3. Префиксные коды обеспечивают однозначное декодирование.

Формат обеспечивает восстановление данных без потерь и является стандартным подходом, используемым в архиваторах (ZIP), системах хранения данных и сетевых протоколах.